

Data Structures and Algorithms Analysis Lab: Lab - 1 Exercise

Submitted By: Patsamatla Bhaavya Ramita

Reg. No: 25MCB1010

Submitted on: 24-07-25

Q1. In a busy airport, passengers are holding tickets with unique numbers. Write a program using the linear search algorithm to check if a given ticket number exists in the array of ticket numbers. If the ticket number is found, print a congratulatory message along with the position where it's found. If the ticket number is not found, encourage the passenger with a message for better luck on the next attempt.

```
#include <stdio.h>
#include <stdbool.h>
int main() {
    int test;
    printf("Enter the number of test cases: ");
    scanf("%d", &test);
    for(int t=0; t<test; t++) {
        int n, x, pos;
        bool flag = false;
        printf("Enter the number of tickets: ");
        scanf("%d", &n);
        int tickets[n];
        for(int i=0; i<n; i++) {
            scanf("%d", &tickets[i]);
        }
        printf("Enter the ticket number to search: ");
        scanf("%d", &x);
        for(int j=0; j<n; j++) {
            if(tickets[j] == x) {
                flag = true;
                pos = j+1;
                break;
            }
        }
        if(flag) {
            printf("Congratulations! Ticket number %d found at %d\n", x, pos);
        }
        else {
            printf("Better luck next time! Ticket number %d not found!\n", x);
        }
    }
}
```

```

    }
}
return 0;
}

```

Output:

```

PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks> cd "c:\Users\Bhaavya\Desktop\VIT\DSALabTasks"
Enter the number of test cases: 2
Enter the number of tickets: 5
123 456 789 321 654
Enter the ticket number to search: 321
Congratulations! Ticket number 321 found at 4
Enter the number of tickets: 6
741 456 982 364 789 258
Enter the ticket number to search: 369
Better luck next time! Ticket number 369 not found!
PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks>

```

Time Complexity: $O(T \times n)$, Space Complexity: $O(n)$

Q2. You are working as a data analyst for a company that collects user feedback scores for their products. The feedback scores are recorded sequentially, and sometimes you need to identify specific feedback points for analysis. Given a list of feedback scores and a particular feedback score, your task is to find the position of the second occurrence of this feedback score in the list. If the feedback score appears less than twice, you need to handle that as well.

```

#include <stdio.h>

int main() {
    int test;
    printf("Enter number of test cases: ");
    scanf("%d", &test);
    for(int t=0; t<test; t++) {
        int n;
        printf("Enter number of feedback scores: ");
        scanf("%d", &n);
        int scores[n];
        int x;
        int count = 0;
        int j;
        for(int i=0; i<n; i++) {

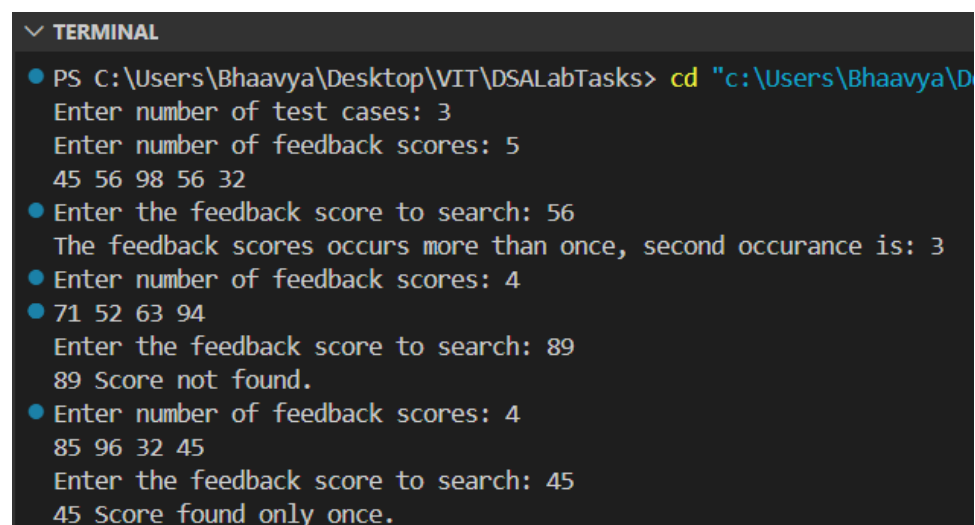
```

```

        scanf("%d", &scores[i]);
    }
    printf("Enter the feedback score to search: ");
    scanf("%d", &x);
    for(j=0; j<n; j++) {
        if(scores[j] == x) {
            count += 1;
        }
        if(count > 1) {
            break;
        }
    }
    if(count > 1) {
        printf("The feedback scores occurs more than once, second occurrence is: %d\n", j);
    }
    else if(count == 1) {
        printf("%d Score found only once.\n", x);
    }
    else if(count == 0) {
        printf("%d Score not found.\n", x);
    }
}
return 0;
}

```

Output:



```

▼ TERMINAL
● PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks> cd "c:\Users\Bhaavya\D
Enter number of test cases: 3
Enter number of feedback scores: 5
45 56 98 56 32
● Enter the feedback score to search: 56
The feedback scores occurs more than once, second occurrence is: 3
● Enter number of feedback scores: 4
● 71 52 63 94
Enter the feedback score to search: 89
89 Score not found.
● Enter number of feedback scores: 4
85 96 32 45
Enter the feedback score to search: 45
45 Score found only once.

```

Time Complexity: $O(T \times n)$, Space Complexity: $O(n)$

Q3. In a local election, each candidate is assigned a unique numeric ID. After collecting the votes, you need to determine if any candidate received more than $n-2$ votes, where n is the total number of votes cast. This will help in identifying if there is a clear winner with significant support. Your task is to write a program that determines if there is a candidate ID that received more than $n-2$ votes. If such a candidate exists, return their ID. Otherwise, return "No majority candidate found."

```
#include <stdio.h>

int main() {
    int test;
    printf("Enter number of test cases: ");
    scanf("%d", &test);
    for(int t=0; t<test; t++) {
        int n;
        printf("Enter number of votes: ");
        scanf("%d", &n);
        int votes[n], count[101] = {0};
        printf("Enter %d candidate IDs: ", n);
        for (int i = 0; i < n; i++) {
            scanf("%d", &votes[i]);
            count[votes[i]]++;
        }
        int majority = 0;
        for (int i = 1; i <= 100; i++) {
            if (count[i] > n - 2) {
                printf("%d\n", i);
                majority = 1;
                break;
            }
        }
        if (!majority) {
            printf("No majority candidate found.\n");
        }
    }
    return 0;
}
```

Output:

```
✓ TERMINAL
● PS C:\Users\Bhaavya> cd "c:\Users\Bhaavya\Desktop\VT
Enter number of test cases: 3
Enter number of votes: 5
Enter 5 candidate IDs: 1 4 1 1 1
1
Enter number of votes: 8
Enter 8 candidate IDs: 12 12 12 12 13 12 12 12
12
Enter number of votes: 5
Enter 5 candidate IDs: 56 58 45 54 55
No majority candidate found.
```

Time Complexity: $O(T \times n)$, Space Complexity: $O(n)$

Q4. You are part of a legal team responsible for reviewing contracts and legal documents. To streamline the review process, you need a tool that can quickly check if specific legal terms or clauses are included in a large text document. This tool helps ensure that essential terms are not overlooked. Write a program that searches a given document (a sentence) for the presence of a specific legal term or clause. The tool should indicate whether the term is present in the document.

```
#include <stdio.h>
#include <string.h>
int main() {
    int test;
    printf("Enter number of test cases: ");
    scanf("%d", &test);
    getchar();
    for(int i=0; i<test; i++) {
        char str[1000];
        char term[100];
        printf("Enter the sentence: ");
        fgets(str, sizeof(str), stdin);
        printf("Enter the legal term to find: ");
        fgets(term, sizeof(term), stdin);
```

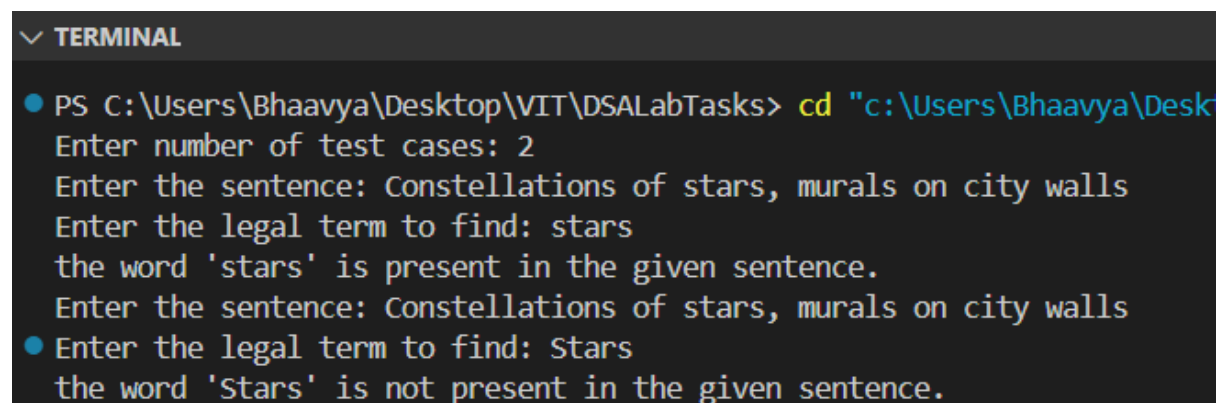
```

str[strcspn(str, "\n")] = '\0';
term[strcspn(term, "\n")] = '\0';

if(strstr(str, term) != NULL) {
    printf("the word '%s' is present in the given sentence.\n", term);
}
else {
    printf("the word '%s' is not present in the given sentence.\n", term);
}
}
return 0;
}

```

Output:



```

▼ TERMINAL
● PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks> cd "c:\Users\Bhaavya\Desktop"
Enter number of test cases: 2
Enter the sentence: Constellations of stars, murals on city walls
Enter the legal term to find: stars
the word 'stars' is present in the given sentence.
Enter the sentence: Constellations of stars, murals on city walls
● Enter the legal term to find: Stars
the word 'Stars' is not present in the given sentence.

```

Time Complexity: $O(T \times L \times k)$, Space Complexity: $O(l \times k)$

Where l = length of string, k = length of term

Q5. In a warehouse, inventory management requires checking the availability of specific items in stock. Each item in the inventory is identified by a unique numerical ID. Given a list of items currently in the warehouse and a set of item IDs that need to be checked, determine if each item ID from the set is present in the warehouse inventory and identify its position if it is available. Implement a tool that reads the inventory IDs, followed by a list of item IDs to be checked, and then performs a search to determine if each item ID is present in the inventory. For each item ID, output its position in the inventory if found, or indicate that the item is not available.

```

#include <stdio.h>
#include <stdbool.h>

int main() {

```

```

int test;
printf("Enter number of test cases: ");
scanf("%d", &test);
getchar();
for(int i=0; i<test; i++) {
    int n1;
    printf("Enter the number of items in inventory: ");
    scanf("%d", &n1);
    getchar();
    int items[n1];
    printf("Enter the IDs of items: ");
    for(int i=0; i<n1; i++) {
        scanf("%d", &items[i]);
    }
    int n2;
    printf("Enter the number of IDs to be checked: ");
    scanf("%d", &n2);
    getchar();
    int check;
    bool flag;
    printf("Enter the IDs to be checked: ");
    for(int j=0; j<n2; j++) {
        scanf("%d", &check);
        flag = false;
        for(int k=0; k<n1; k++) {
            if(check == items[k]) {
                printf("%d found at position %d\n", check, k+1);
                flag = true;
            }
        }
        if(!flag) {
            printf("%d not found in the inventory\n", check);
        }
    }
}
return 0;

```

}

Output:

```
▼ TERMINAL
● PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks> cd "c:\User
Enter number of test cases: 2
Enter the number of items in inventory: 5
Enter the IDs of items: 10 20 30 40 50
Enter the number of Ids to be checked: 4
● Enter the IDs to be checked: 20 30 40 60
20 found at position 2
30 found at position 3
● 40 found at position 4
60 not found in the inventory
Enter the number of items in inventory: 4
Enter the IDs of items: 11 22 33 44
Enter the number of Ids to be checked: 2
Enter the IDs to be checked: 22 55
22 found at position 2
55 not found in the inventory
```

Time Complexity: $O(T \times n_1 \times n_2)$, Space Complexity: $O(n_1)$

Q6. In a company's customer service center, each customer interaction is logged with a unique numerical ID. A customer support representative needs to search through these logs to find the position of a specific customer interaction ID and also analyze the overall interaction data. Implement a tool to search for a specific customer interaction ID in a log of interaction IDs, determine its position if it exists, and report the total number of comparisons made during the search. Additionally, calculate and display the total number of interactions and the average number of interactions per ID.

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int test;
    printf("Enter number of test cases: ");
    scanf("%d", &test);
    getchar();
    for(int i=0; i<test; i++) {
        int n;
        printf("Enter the number of customer interactions: ");
```



```

scanf("%d", &n);
getchar();
int ids[n];
printf("Enter the interaction IDs: ");
for(int i=0; i<n; i++) {
    scanf("%d", &ids[i]);
}
int x;
printf("Enter the interaction id to be searched: ");
scanf("%d", &x);
getchar();
int count = 0;
int sum = 0;
float avg = 0.0;
bool flag = false;
int cmp;
for(int j=0; j<n; j++) {
    if(ids[j] == x) {
        printf("Interaction ID %d is present at index %d\n", x, j);
        flag = true;
        cmp = count + 1;
    }
    count += 1;
    sum += ids[j];
}
avg = sum/n;
if(!flag) {
    printf("Intercation ID %d is not present\n", x);
    cmp = count;
}
printf("Total comparisions made: %d\n", cmp);
printf("Sum of all interactions: %d\n", sum);
printf("Average of all intercatations: %.2f\n", avg);
}
return 0;
}

```

Output:

```
▼ TERMINAL
● PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks> cd "c:\Us
Enter number of test cases: 2
Enter the number of customer interactions: 5
Enter the interaction IDs: 40 20 50 30 10
Enter the interaction id to be searched: 30
Interaction ID 30 is present at index 3
Total comparisons made: 4
● Sum of all interactions: 150
Average of all intercatations: 30.00
Enter the number of customer interactions: 5
Enter the interaction IDs: 40 20 50 30 10
Enter the interaction id to be searched: 80
Intercation ID 80 is not present
Total comparisons made: 5
Sum of all interactions: 150
Average of all intercatations: 30.00
```

Time Complexity: $O(T \times n)$, Space Complexity: $O(n)$

Q7. Sathish is developing a simple inventory management system for a store. The store owner wants to update the stock levels of certain products based on the latest shipment. Write a program that takes the current stock levels as input, along with the product code to search for and the new quantity received. Implement a linear search program to find the product in the inventory and update its stock with the new quantity. Finally, print the updated inventory after the stock update.

```
#include <stdio.h>
#include <stdbool.h>
int main() {
    int test;
    printf("Enter number of test cases: ");
    scanf("%d", &test);
    getchar();
    for(int i=0; i<test; i++) {
        int n;
        printf("Enter the size of inventory: ");
        scanf("%d", &n);
        getchar();
        int stock[n];
        printf("Enter the stock levels: ");
        for(int i=0; i<n; i++) {
```

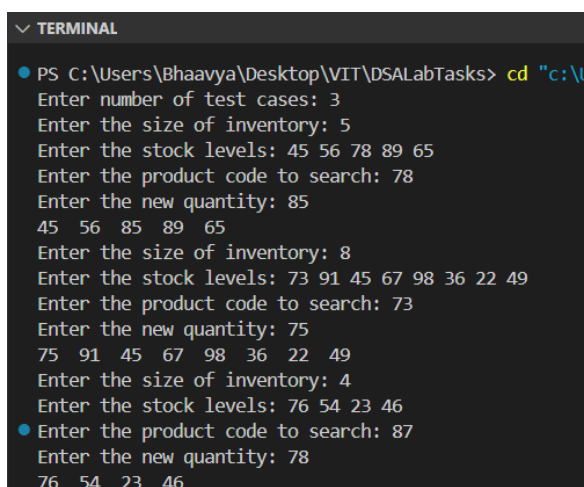
```

        scanf("%d", &stock[i]);
    }
    int x;
    printf("Enter the product code to search: ");
    scanf("%d", &x);
    getchar();
    int m;
    printf("Enter the new quantity: ");
    scanf("%d", &m);
    getchar();

    for(int j=0; j<n; j++) {
        if(stock[j] == x) {
            stock[j] = m;
        }
        printf("%d ", stock[j]);
        if(j != n-1) {
            printf(" ");
        }
    }
    printf("\n");
}
return 0;
}

```

Output:



```

▼ TERMINAL
● PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks> cd "c:\V
Enter number of test cases: 3
Enter the size of inventory: 5
Enter the stock levels: 45 56 78 89 65
Enter the product code to search: 78
Enter the new quantity: 85
45 56 85 89 65
Enter the size of inventory: 8
Enter the stock levels: 73 91 45 67 98 36 22 49
Enter the product code to search: 73
Enter the new quantity: 75
75 91 45 67 98 36 22 49
Enter the size of inventory: 4
Enter the stock levels: 76 54 23 46
● Enter the product code to search: 87
Enter the new quantity: 78
76 54 23 46

```

Time Complexity: $O(T \times n)$, Space Complexity: $O(n)$

Q8. In a manufacturing plant, quality control inspectors need to verify that all parts on a production line meet specific quality standards. Each part is assigned an ID number, and for the parts to be approved, all IDs must be even numbers. Your task is to develop a tool that helps in checking whether all parts in a batch meet this requirement. Given a list of part IDs, determine if every ID in the list is an even number. If all IDs are even, report that the batch is approved; otherwise, report that the batch is not approved.

```
#include <stdio.h>
#include <stdbool.h>
int main() {
    int test;
    printf("Enter number of test cases: ");
    scanf("%d", &test);
    getchar();
    for(int i=0; i<test; i++) {
        bool flag = false;
        int n;
        printf("Enter the size of list: ");
        scanf("%d", &n);
        getchar();
        int id;
        printf("Enter the part IDs: ");
        for(int i=0; i<n; i++) {
            scanf("%d", &id);
            if(id % 2 != 0) {
                flag = true;
            }
        }
        if(flag) {
            printf("The batch is not approved\n");
        }
        else {
            printf("All parts in the batch are approved\n");
        }
    }
    return 0;
}
```

Output:

```
✓ TERMINAL
● PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks> cd "
Enter number of test cases: 2
Enter the size of list: 5
● Enter the part IDs: 12 18 24 16 38
All parts in the batch are approved
Enter the size of list: 3
Enter the part IDs: 15 26 39
The batch is not approved
```

Time Complexity: $O(T \times n)$, Space Complexity: $O(1)$

Q9 Alex, a logistics manager, is responsible for handling various pieces of luggage. He needs to determine the weight distribution among these pieces and find the heaviest and lightest pieces, along with the weight difference between them. To accomplish this, he plans to create a simple program that takes as input the weights of various pieces of luggage and uses the **linear search** algorithm to search and identify the heaviest and lightest pieces, as well as calculate the weight difference between them. Write a program to help Alex accomplish this task.

```
#include <stdio.h>

int main() {
    int test;
    printf("Enter number of test cases: ");
    scanf("%d", &test);
    getchar();
    for(int i=0; i<test; i++) {
        int n;
        printf("Enter the number of luggage: ");
        scanf("%d", &n);
        getchar();
        int weight[n];
        printf("Enter the weights of luggage: ");
        for(int i=0; i<n; i++) {
            scanf("%d", &weight[i]);
        }
        int max = weight[0];
```

```

int min = weight[0];
for(int j=0; j<n; j++) {
    if(max < weight[j]) {
        max = weight[j];
    }
    if (min > weight[j]) {
        min = weight[j];
    }
}
printf("Heaviest piece weight: %d\n", max);
printf("Lighest piece weight: %d\n", min);
printf("Weight difference: %d\n", max-min);
}
return 0;
}

```

Output:

```

● IT\DSALabTasks\" ; if ($?) { gcc task9.c -o task9 } ; if ($?) { .\task9 }
Enter number of test cases: 2
Enter the number of luggage: 5
● Enter the weights of luggage: 10 15 20 25 30
Heaviest piece weight: 30
Lighest piece weight: 10
Weight difference: 20
● Enter the number of luggage: 8
● Enter the weights of luggage: 70 65 80 75 60 55 50 45
Heaviest piece weight: 80
Lighest piece weight: 45
Weight difference: 35

```

Time Complexity: $O(T \times n)$, Space Complexity: $O(n)$

Q10. In a research lab specializing in data collection, the team frequently deals with large datasets where quick lookup and data reversal are crucial. The team needs to ensure that certain data points (identified by unique IDs) are present in their datasets and also needs to analyze data trends by reversing the order of the collected data.

```

#include <stdio.h>
#include <stdbool.h>
int main() {

```

```

int test;
printf("Enter number of test cases: ");
scanf("%d", &test);
getchar();
for(int i=0; i<test; i++) {
    int n;
    printf("Enter the number of data points: ");
    scanf("%d", &n);
    getchar();
    int data[n];
    printf("Enter the data points: ");
    for(int i=0; i<n; i++) {
        scanf("%d", &data[i]);
    }
    int x;
    printf("Enter the data point to be searched: ");
    scanf("%d", &x);
    getchar();
    bool flag = false;
    int rev[n];
    for(int j=0; j<n; j++) {
        if(data[j] == x){
            printf("Data point is present in the dataset at position %d\n", j);
            flag = true;
        }
        rev[j] = data[n-j-1];
    }
    if(!flag) {
        printf("Data point is not present in the dataset\n");
    }
    for(int k=0; k<n; k++) {
        printf("%d ", rev[k]);
    }
    printf("\n");
}
return 0;

```

```
}
```

Output:

```
PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks> cd "c:\U
● Enter number of test cases: 2
● Enter the number of data points: 5
  Enter the data points: 45 56 98 97 32
  Enter the data point to be searched: 56
  Data point is present in the dataset at position 1
● 32 97 98 56 45
  Enter the number of data points: 4
  Enter the data points: 71 52 63 94
  Enter the data point to be searched: 89
  Data point is not present in the dataset
  94 63 52 71
```

Time Complexity: $O(T \times n)$, Space Complexity: $O(n)$

Q11. In a large-scale sales database, the management team is tasked with performing critical data analysis to ensure optimal business decisions. The database contains records of sales amounts for various transactions. For strategic planning, the team needs to: Quickly check if a particular sales amount has been recorded and, if so, identify its position in the dataset. To understand competitive performance and identify the second-best sales figures, which may be crucial for benchmarking and incentive planning.

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>
int main() {
    int test;
    printf("Enter number of test cases: ");
    scanf("%d", &test);
    getchar();
    for(int i=0; i<test; i++) {
        int n;
        printf("Enter the number of transactions: ");
        scanf("%d", &n);
        getchar();
        int data[n];
        printf("Enter the sales amounts: ");
```



```

for(int i=0; i<n; i++) {
    scanf("%d", &data[i]);
}
int x;
printf("Enter the sales amount to be searched: ");
scanf("%d", &x);
getchar();
bool flag = false;
int first = INT_MIN;
int second = INT_MIN;
for(int j=0; j<n; j++) {
    if(data[j] == x){
        printf("Sales amount %d is recorded at position %d\n", x, j+1);
        flag = true;
    }
    if(data[j] > first) {
        second = first;
        first = data[j];
    }
    else if (data[j] > second && data[j] != first) {
        second = data[j];
    }
}
if(!flag) {
    printf("Sales amount %d is not recorded in the dataset\n", x);
}
if (second != INT_MIN) {
    printf("%d\n", second);
}
else {
    printf("No second highest sales amount found\n");
}
}
return 0;
}

```

Output:

```

PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks> cd "c:\U
Enter number of test cases: 3
Enter the number of transactions: 5
Enter the sales amounts: 19 15 24 43 37
Enter the sales amount to be searched: 16
● Sales amount 16 is not recorded in the dataset
37
Enter the number of transactions: 5
Enter the sales amounts: 19 15 24 43 37
Enter the sales amount to be searched: 24
Sales amount 24 is recorded at position 3
37
Enter the number of transactions: 1
Enter the sales amounts: 2
Enter the sales amount to be searched: 3
Sales amount 3 is not recorded in the dataset
No second highest sales amount found

```

Time Complexity: $O(T \times n)$, Space Complexity: $O(n)$

Q12. Arsh is working on enhancing a recommendation system for a shopping application. He needs to develop a feature that suggests the closest product prices to a target price entered by a user. The list of product prices is sorted in non-decreasing order, and he wants the system to find the K closest prices to the target price. If two prices have the same difference from the target, the lower price should be preferred. To implement this feature, Arsh needs to write a program that identifies and prints the K closest prices to the given target price from the sorted list of prices. If there are two prices with the same difference value, print the larger price.

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int test;
    printf("Enter the number of test cases: ");
    scanf("%d", &test);
    for(int t=0; t<test; t++) {
        int n;
        printf("Enter the number of product prices: ");
        scanf("%d", &n);
        getchar();
        int prices[n];
        for(int i=0; i<n; i++) {
            scanf("%d", &prices[i]);
        }
    }
}

```

```

int x;
printf("Enter target price: ");
scanf("%d", &x);
getchar();

int k;
printf("Enter number of closest prices to find: ");
scanf("%d", &k);
getchar();


int num[n];
for(int j=0; j<n; j++) {
    num[j] = abs(prices[j] - x);
}
for(int k=0; k < n-1; k++) {
    for(int y=k+1; y<n; y++) {
        if((num[k] > num[y]) || (num[k] == num[y] && prices[k] < prices[y])) {
            int temp = num[k] ;
            num[k] = num[y];
            num[y] = temp;
            temp = prices[k];
            prices[k] = prices[y];
            prices[y] = temp;
        }
    }
}
for(int i=0; i<k; i++) {
    printf("%d ", prices[i]);
}
printf("\n");
}
return 0;
}

```

Output:

```
• }
  Enter the number of test cases: 2
  Enter the number of product prices: 5
• 2 4 7 8 10
  Enter target price: 5
  Enter number of closest prices to find: 3
  4 7 8
  Enter the number of product prices: 8
• 1 3 5 7 9 11 13 15
  Enter target price: 10
  Enter number of closest prices to find: 1
  11
○ PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks> █
```

Time Complexity: $O(T \times n^2)$, Space Complexity: $O(n)$

Q13. You are in charge of preparing a series of meals for a special event. Each meal needs to be cooked using ingredients from a set of ingredient boxes. Each box becomes available on a specific day, and you need to cook a certain number of meals using the ingredients from adjacent boxes. You are given an integer array availabilityDay, an integer m, and an integer k. availabilityDay[i] represents the day when the i-th ingredient box becomes available. To prepare one meal, you need to use k adjacent ingredient boxes. You need to prepare m meals. Your task is to find the minimum number of days you need to wait to be able to prepare m meals using the available ingredient boxes. If it's impossible to prepare the required number of meals, return -1. Implement the problem using recursive binary search.

```
#include <stdio.h>
#include <stdbool.h>
int availabilityDay[100];
bool canCook(int day, int n, int m, int k) {
    int count = 0, boxes = 0;

    for (int i = 0; i < n; i++) {
        if (availabilityDay[i] <= day) {
            boxes++;
            if (boxes == k) {
                count++;
                boxes = 0;
            }
        }
    }
}
```

```

    } else {
        boxes = 0;
    }
}
return count >= m;
}

int main() {
    int testCases;
    printf("Enter number of test cases: ");
    scanf("%d", &testCases);

    for (int t = 0; t < testCases; t++) {
        int n, m, k;
        printf("Enter number of ingredient boxes: ");
        scanf("%d", &n);

        printf("Enter availability days: ");
        for (int i = 0; i < n; i++) {
            scanf("%d", &availabilityDay[i]);
        }
        printf("Enter number of meals required: ");
        scanf("%d", &m);
        printf("Enter number of adjacent boxes per meal: ");
        scanf("%d", &k);

        if (m * k > n) {
            printf("-1\n");
            continue;
        }
        int maxDay = 0;
        for (int i = 0; i < n; i++) {
            if (availabilityDay[i] > maxDay)
                maxDay = availabilityDay[i];
        }
        int found = -1;
        for (int day = 1; day <= maxDay; day++) {

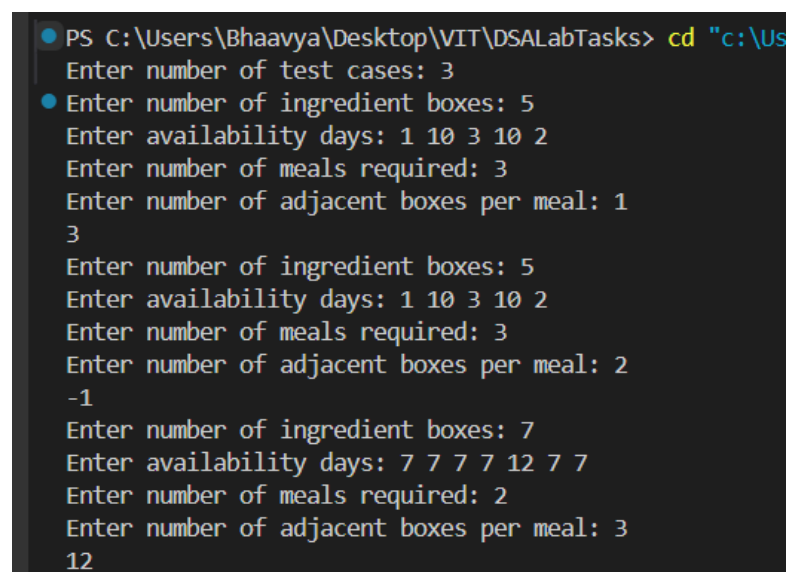
```

```

        if (canCook(day, n, m, k)) {
            found = day;
            break;
        }
    }
    printf("%d\n", found);
}
return 0;
}

```

Output:



```

PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks> cd "c:\Us
Enter number of test cases: 3
Enter number of ingredient boxes: 5
Enter availability days: 1 10 3 10 2
Enter number of meals required: 3
Enter number of adjacent boxes per meal: 1
3
Enter number of ingredient boxes: 5
Enter availability days: 1 10 3 10 2
Enter number of meals required: 3
Enter number of adjacent boxes per meal: 2
-1
Enter number of ingredient boxes: 7
Enter availability days: 7 7 7 7 12 7 7
Enter number of meals required: 2
Enter number of adjacent boxes per meal: 3
12

```

Time Complexity: $O(T \times n)$, Space Complexity: $O(n)$

Q.14 You are part of a team managing an amusement park with several rides, each identified by a unique ride ID. These ride IDs are stored in a sorted list in ascending order. When a new ride is introduced or when the system needs to locate a ride, you need to find the correct position of the ride ID in the list. If the ride ID already exists, return its position. If not, return the index where the new ride ID should be inserted to maintain the order.

```

#include <stdio.h>
#include <stdbool.h>
int main() {
    int test;
    printf("Enter the number of test cases: ");
    scanf("%d", &test);
    for(int t=0; t<test; t++) {

```

```

int n;
printf("Enter number of existing ride IDs: ");
scanf("%d", &n);
getchar();
int ids[n];
printf("Enter the IDs in ascending order: ");
for(int i=0; i<n; i++) {
    scanf("%d", &ids[i]);
}
int targetId;
bool flag = false;
printf("Enter the targetId: ");
scanf("%d", &targetId);
getchar();
for(int i=0; i<n; i++) {
    if(targetId <= ids[i]) {
        printf("%d\n", i);
        flag = true;
        break;
    }
}
if(!flag) {
    printf("%d\n", n);
}
}
}

```

```

PS C:\Users\Bhaavya\Desktop\VIT\DSALabTasks> cd
Enter the number of test cases: 4
Enter number of existing ride IDs: 4
Enter the IDs in ascending order: 1 3 5 6
Enter the targetId: 5
2
Enter number of existing ride IDs: 4
Enter the IDs in ascending order: 1 3 5 6
Enter the targetId: 2
1
Enter number of existing ride IDs: 3
Enter the IDs in ascending order: 10 20 30
Enter the targetId: 40
3
Enter number of existing ride IDs: 3
Enter the IDs in ascending order: 10 20 40
Enter the targetId: 1
0

```

Time Complexity: $O(T \times n)$, Space Complexity: $O(n)$

Q15. You are working as a software developer at a financial analytics company. The company provides tools for various financial analyses, including computing statistics on sets of data. Today, you are assigned to enhance a tool that calculates the median of two datasets. These datasets represent financial transactions recorded over different periods. Your task is to develop a feature that finds the median of the combined data, which helps in understanding the central tendency of transaction amounts over time.

```
#include <stdio.h>

int main() {
    int test;
    printf("Enter the number of test cases: ");
    scanf("%d", &test);
    for(int t=0; t<test; t++) {
        int n1, n2;
        printf("Enter the number of data points of first dataset: ");
        scanf("%d", &n1);
        getchar();
        int first[n1];
        printf("Enter the data points: ");
        for(int i=0; i<n1; i++) {
            scanf("%d", &first[i]);
        }
        printf("Enter the number of data points of second dataset: ");
        scanf("%d", &n2);
        getchar();
        int second[n2];
        printf("Enter the data points: ");
        for(int i=0; i<n2; i++) {
            scanf("%d", &second[i]);
        }

        int n = n1+n2;
        int merge[n];
        for(int i=0; i<n1; i++) {
            merge[i] = first[i];
        }
```



```

for(int j=0; j<n2; j++) {
    merge[n1+j] = second[j];
}
for (int i = 0; i < n-1; i++) {
    for (int j = 0; j < n-i-1; j++) {
        if (merge[j] > merge[j + 1]) {
            int temp = merge[j];
            merge[j] = merge[j + 1];
            merge[j + 1] = temp;
        }
    }
}
float median;
if(n%2 == 1) {
    median = merge[n/2];
}
else {
    median = (merge[n/2 - 1] + merge[n/2]) / 2.0;
}
printf("%.2f\n", median);
}
}

```

Output:

```

IT\DSALabTasks\" ; if ($?) { gcc task15.c -o task15 } ; i
Enter the number of test cases: 3
Enter the number of data points of first dataset: 3
Enter the data points: 10 20 30
Enter the number of data points of second dataset: 2
Enter the data points: 10 20
20.00
Enter the number of data points of first dataset: 5
Enter the data points: 1 2 3 4 5
Enter the number of data points of second dataset: 2
Enter the data points: 6 7
4.00
Enter the number of data points of first dataset: 3
Enter the data points: 54 87 90
Enter the number of data points of second dataset: 1
Enter the data points: 67
77.00

```

Time Complexity: $O(T \times n^2)$, Space Complexity: $O(n)$